

Preparação de um Projeto com OpenSpec

Desenvolvimento de software com apoio de agentes de Inteligência Artificial

Guia para criação do contexto, documentação e configuração inicial

1. Objetivo

Este manual apresenta o procedimento para preparar um projeto de software para ser desenvolvido com apoio de um **agente de Inteligência Artificial utilizando OpenSpec**.

Cada grupo desenvolverá um projeto com um **tema próprio**. Portanto, este manual não apresenta um domínio específico, não define funcionalidades e não pressupõe uma determinada arquitetura de software.

O objetivo é ensinar o grupo a construir a **fundação documental e de contexto** necessária para que um agente de IA consiga compreender o projeto antes de começar a implementá-lo.

Ao final deste manual, o projeto deverá possuir:

projeto/

|

|— AGENTS.md

|

|— docs/

| |— project-overview.md

| |— domain-model.md

|

```
└─ openspec/  
    └─ config.yaml  
    └─ specs/  
        └─ changes/  
            └─ archive/
```

Além disso, o OpenSpec deverá estar inicializado na ferramenta de IA utilizada pelo grupo.

Atenção: alguns dos arquivos apresentados neste manual são convenções adotadas para o projeto acadêmico. Eles não são todos arquivos obrigatórios do OpenSpec. O OpenSpec possui sua própria estrutura, especialmente a pasta `openspec/` e o arquivo `openspec/config.yaml`.

2. O princípio fundamental

O desenvolvimento deverá seguir o princípio:

Primeiro fornecer contexto suficiente para a IA compreender o projeto; depois desenvolver o sistema de forma incremental.

Isso não significa escrever toda a especificação do sistema antes de começar.

Também não significa simplesmente entregar uma descrição vaga ao agente e pedir:

"Construa meu sistema."

O objetivo é estabelecer uma **visão global suficientemente clara** para que o agente consiga tomar decisões coerentes, e utilizando o OpenSpec, conduzir o desenvolvimento em mudanças menores.

O processo pode ser representado assim:

Tema do projeto

|



Visão geral



Modelo do domínio



Instruções para o agente



Configuração do OpenSpec



Explore



Primeira Change



Review



Apply



Archive



Próxima Change

3. Por que fornecer contexto antes de implementar?

Um agente de IA consegue gerar código rapidamente.

O problema é que **gerar código não é o mesmo que compreender o projeto**.

Sem contexto suficiente, o agente pode:

- interpretar incorretamente o objetivo do sistema;
- criar estruturas incompatíveis entre si;
- duplicar funcionalidades;
- criar um banco de dados incompatível com o domínio;
- implementar regras de negócio no lugar errado;
- tomar decisões arquiteturais que dificultem mudanças futuras.

Os documentos de contexto existem para reduzir esse problema.

Eles devem responder, em níveis diferentes, às perguntas:

O que estamos construindo?



Quais são os conceitos do sistema?



Como o agente deve trabalhar?



Quais informações o OpenSpec deve considerar
em todas as mudanças?

4. Os documentos e suas funções

Neste projeto serão utilizados quatro documentos principais de contexto:

Documento	Pergunta que responde
<code>project-overview.md</code>	O que é o projeto?
<code>domain-model.md</code>	Quais são os principais conceitos e relacionamentos?
<code>AGENTS.md</code>	Como o agente deve trabalhar neste projeto?
<code>openspec/config.yaml</code>	Que contexto e regras o OpenSpec deve fornecer aos seus workflows?

Além deles, o próprio OpenSpec criará e administrará:

`openspec/specs/`

`openspec/changes/`

5. Quanto detalhe deve ser colocado?

Esta é uma das questões mais importantes deste manual.

O objetivo **não é escrever o máximo possível**.

O objetivo é escrever o **detalhamento adequado para cada documento**.

Uma boa regra é:

Visão geral



mais abrangente e estável

Modelo de domínio



conceitual e estrutural

AGENTS.md



regras de trabalho

config.yaml



curto, estável e diretamente relevante aos workflows

specs/



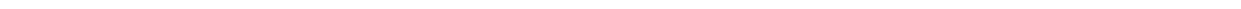
comportamento detalhado e verificável

changes/



detalhes da mudança atual

Quanto mais próximo da implementação, mais específico o documento pode ser.



6. Criar o Project Overview

Crie o arquivo:

docs/project-overview.md

Você obterá:

projeto/

|

└─ docs/

 └─ project-overview.md

Esse documento deve apresentar o projeto para alguém que **não participou das discussões anteriores**.

Imagine que outra pessoa abra o repositório pela primeira vez e leia somente esse arquivo.

Ela deverá conseguir compreender:

- qual problema o sistema resolve;
- quem utilizará o sistema;
- qual é o objetivo;
- qual é o escopo inicial;
- quais são as principais funcionalidades esperadas;
- quais são as principais restrições;
- quais tecnologias ou princípios já foram definidos;
- quais aspectos são considerados críticos.

7. O que colocar no Project Overview

Recomenda-se utilizar uma estrutura semelhante a:

Project Overview — [Nome do Projeto]

1. Visão geral

2. Problema

3. Objetivos

4. Público-alvo / usuários

5. Escopo

6. Principais funcionalidades

7. Requisitos e restrições importantes

8. Arquitetura tecnológica

9. Princípios de desenvolvimento

10. Segurança e integridade

11. Estratégia de desenvolvimento

12. Fonte de verdade e documentação

Nem todas as seções são obrigatórias.

Adapte a estrutura ao projeto.

8. Nível de detalhamento do Project Overview

O documento deve ser **suficientemente detalhado para transmitir a visão do projeto**, mas não precisa especificar cada comportamento.

Por exemplo, em um sistema hipotético de biblioteca:

Bom nível de detalhe

O sistema permitirá que usuários consultem o catálogo,

realizem empréstimos e acompanhem seus empréstimos ativos.

Bibliotecários poderão cadastrar livros, exemplares e usuários,

além de controlar empréstimos e devoluções.

Detalhamento excessivo para esse documento

Quando o usuário clicar no botão "Emprestar", o frontend

deverá executar `POST /api/loans/create`, enviando...

Esse segundo nível pertence à especificação ou ao design de uma mudança específica.

9. O que NÃO colocar no Project Overview

Evite transformar o documento em:

- documentação de API;
- manual de programação;
- modelo físico do banco;
- lista de endpoints;
- especificação detalhada de cada tela;
- checklist de implementação.

O `project-overview.md` deve permanecer relativamente estável.

10. Criar o Domain Model

Crie o arquivo:

`docs/domain-model.md`

Você obterá:

`projeto/`

|

└─ docs/

|— project-overview.md

└— domain-model.md

Esse documento descreve os **conceitos fundamentais do domínio do projeto e seus relacionamentos**.

Ele deve responder:

"Quais são as coisas importantes que existem neste sistema e como elas se relacionam?"

Por exemplo, em um sistema hipotético de biblioteca:

Usuário

|

|— Empréstimo

| |

| └— Exemplar

| |

| └— Livro

|

└— Reserva

O modelo deve representar os conceitos, não necessariamente as tabelas.

11. O que colocar no Domain Model

Para cada entidade ou conceito importante, procure registrar:

Nome

Descrição

Responsabilidade

Principais atributos conceituais

Relacionamentos

Regras estruturais importantes

Por exemplo:

Livro

Representa uma obra disponível no acervo.

Principais informações

- título;
- autor;
- ISBN;
- categoria.

Relacionamentos

Um livro pode possuir vários exemplares.

Um exemplar pertence a um único livro.

12. Modelo conceitual ≠ banco de dados

Não é necessário determinar inicialmente:

CREATE TABLE ...

nem necessariamente:

livros

id INTEGER PRIMARY KEY

...

O modelo conceitual deve permitir que o agente compreenda o domínio antes de decidir todos os detalhes físicos.

Entretanto, se o projeto já possui decisões importantes sobre persistência, elas podem ser registradas em nível apropriado.

13. Por que criar o modelo global antes de implementar?

Imagine que um grupo tenha que implementar uma funcionalidade de "cadastro de pedido".

Sem uma visão global, o agente pode criar:

Pedido

Cliente

Produto

de maneira isolada.

Depois pode surgir a necessidade de:

Pagamento

Entrega

ItemPedido

Estoque

NotaFiscal

e algumas decisões iniciais podem precisar ser refeitas.

O modelo de domínio permite ao agente enxergar:

Cliente

↓

Pedido

|— ItemPedido

| ↓

| Produto

|

|— Pagamento

|

|— Entrega

Isso **não significa implementar todas essas entidades imediatamente.**

Significa compreender o sistema antes de tomar decisões locais.

14. Criar o AGENTS.md

Crie na raiz o arquivo:

AGENTS.md

Você obterá:

projeto/

|

|— AGENTS.md

|

|— docs/

|— project-overview.md

|— domain-model.md

Esse arquivo contém **instruções para os agentes de IA que trabalham no projeto**.

Enquanto o `project-overview.md` explica o projeto, o `AGENTS.md` explica **como o agente deve atuar dentro dele**.

15. O que deve entrar no AGENTS.md

Inclua regras relativamente estáveis, como:

Documentação

Antes de realizar alterações significativas,

consultar os documentos de contexto do projeto.

Arquitetura

Respeitar as tecnologias definidas no projeto.

Não introduzir tecnologias alternativas sem justificativa.

Código

Reutilizar código existente quando apropriado.

Evitar duplicação.

Não modificar funcionalidades não relacionadas

à mudança atual sem justificativa.

Segurança

Regras de autorização devem ser aplicadas no backend.

Desenvolvimento

Mudanças devem utilizar OpenSpec.

Testes

Mudanças funcionais devem possuir estratégia de verificação.

16. O que não colocar no AGENTS.md

Não transforme o arquivo em uma cópia de toda a documentação do projeto.

Evite repetir:

todos os requisitos;

todas as entidades;

todas as funcionalidades;

todos os detalhes da arquitetura.

O agente poderá consultar os demais documentos.

O **AGENTS.md** deve funcionar como um **conjunto de regras operacionais**.

17. Criar o OpenSpec

Depois de criar os documentos de contexto, inicialize o OpenSpec.

Primeiro, certifique-se de que está na raiz do projeto:

```
cd caminho/do/projeto
```

Depois:

`openspec init`

O `init` inicializa o OpenSpec no projeto e solicita a seleção das ferramentas de IA que serão utilizadas. Ele cria a pasta `openspec/` e instala os workflows correspondentes à ferramenta selecionada.

Após a inicialização, a estrutura básica será:

`openspec/`

├── `config.yaml`

├── `specs/`

└── `changes/`

 └── `archive/`

Os diretórios `specs/` e `changes/` podem estar vazios inicialmente. Isso é normal.

18. Os arquivos criados pelo `init`

O `openspec init` também instala os arquivos de workflow na ferramenta de IA selecionada.

Por exemplo, dependendo da ferramenta, podem ser criados arquivos em diretórios como:

`.agents/`

`.claude/`

ou outro diretório correspondente à ferramenta.

Não altere ou remova esses arquivos sem entender sua finalidade.

Eles são os mecanismos que permitem ao agente executar os workflows do OpenSpec.

O `init` pode ser executado novamente com segurança caso seja necessário adicionar outra ferramenta ou atualizar os arquivos instalados.

19. Configurar o config.yaml

O arquivo:

openspec/config.yaml

é diferente dos documentos anteriores.

Ele não é simplesmente uma documentação do projeto.

A documentação oficial define o `config.yaml` como a configuração que informa aos workflows **como as mudanças devem ser planejadas**, fornecendo `context`, `rules` e orientações para `operations`.

20. O campo `schema`

O arquivo deverá definir o schema utilizado pelo projeto:

schema: spec-driven

O `spec-driven` é o schema padrão do OpenSpec.

Ele define os artefatos produzidos para uma change:

proposal.md

specs/

design.md

tasks.md

e a ordem de suas dependências.

21. O campo **context**

O campo:

context: |

é provavelmente a parte mais importante da configuração inicial.

Ele contém **informações que o agente deve receber durante os workflows do OpenSpec.**

A documentação deixa claro que esse contexto é injetado nas instruções dos artefatos e também em **apply** e **archive**.

22. O que deve entrar no **context**

Inclua informações que:

devem influenciar praticamente todas as mudanças do projeto.

Por exemplo:

context: |

Project: Sistema de gerenciamento de biblioteca.

Domain: Sistema para gerenciamento de livros,
exemplares, usuários e empréstimos.

Tech stack: Python, FastAPI, PostgreSQL e React.

Architecture: Backend responsável por regras de negócio
e persistência. Frontend não deve ser considerado
mecanismo de segurança.

Development: Implementar mudanças incrementalmente

utilizando OpenSpec.

Language: Escrever os artefatos OpenSpec em português brasileiro.

23. Como decidir se uma informação pertence ao **context**

Utilize esta pergunta:

"Essa informação deve ser considerada pelo agente em praticamente todas as mudanças?"

Se sim, pode fazer sentido colocá-la no **context**.

Exemplo

Se todo o projeto obrigatoriamente utiliza:

Python

PostgreSQL

React

isso pode entrar no **context**.

Se apenas uma funcionalidade utiliza:

Auditoria de operações

provavelmente essa informação deve aparecer na change ou no **design.md** correspondente.

24. O **context** não deve copiar toda a documentação

Esse é um erro comum.

Não faça:

context: |

[copiar todo o project-overview.md]

[copiar todo o domain-model.md]

[copiar todo o AGENTS.md]

O **context** é enviado ao agente em praticamente todos os workflows.

Por isso, quanto mais informação irrelevante for colocada ali, maior será o ruído.

A documentação oficial recomenda manter as regras curtas porque elas são adicionadas ao contexto do agente.

25. Nível de detalhamento recomendado para o context

O **context** deve normalmente conter:

Identidade do projeto

Project: ...

Domínio

Domain: ...

Stack

Tech stack: ...

Restrições arquiteturais

Architecture: ...

Regras críticas

Security:

...

Princípios permanentes

Development:

...

Idioma

Language:

...

Isso normalmente é suficiente.

26. O campo **rules**

O campo **rules** permite adicionar regras específicas a determinado artefato.

Por exemplo:

rules:

proposal:

- Manter a proposta concisa.
- Explicitar o que está fora do escopo.

specs:

- Escrever requisitos verificáveis.
- Utilizar cenários para comportamentos importantes.

tasks:

- Dividir as tarefas em etapas pequenas e verificáveis.

A documentação esclarece que essas regras **complementam** as instruções internas do OpenSpec; elas não substituem as instruções padrão.

27. Não exagerar nas rules

Evite criar dezenas de regras.

Por exemplo, isto é ruim:

rules:

tasks:

- Sempre fazer X.
- Nunca fazer Y.
- Sempre verificar Z.
- Fazer A.
- Fazer B.
- Fazer C.
- ...

O objetivo é acrescentar apenas regras realmente importantes para aquele projeto.

Pergunte:

"O comportamento padrão do OpenSpec precisa ser complementado por alguma regra específica do meu projeto?"

Se a resposta for não, não acrescente a regra.

28. O campo `operations`

O `config.yaml` também permite definir orientações para:

`operations:`

`apply:`

`guidance:`

`...`

`archive:`

`guidance:`

`...`

Essas orientações são diferentes das `rules`.

`rules` orientam a criação de determinado artefato.

`operations` orientam a execução de `apply` e `archive`.

Por exemplo:

`operations:`

`apply:`

`guidance:`

- Executar os testes relevantes antes de concluir as tarefas.

`archive:`

`guidance:`

- Confirmar que a implementação corresponde às especificações.

29. Modelo genérico de config.yaml

Cada grupo deverá adaptar o arquivo ao próprio projeto.

Um modelo inicial pode ser:

schema: spec-driven

context: |

Project: [nome do projeto]

Domain: [descrição resumida do domínio]

Tech stack:

[tecnologias definidas para o projeto]

Architecture:

[decisões arquiteturais que devem ser respeitadas

em praticamente todas as mudanças]

Security:

[regras de segurança permanentes]

Development:

O projeto será desenvolvido incrementalmente

utilizando OpenSpec.

Mudanças relevantes devem ser especificadas

antes da implementação.

Language:

Escrever a documentação do projeto e os artefatos

OpenSpec em português brasileiro.

rules:

proposal:

- Descrever claramente o problema, objetivo e escopo.
- Identificar o que está fora do escopo quando necessário.

specs:

- Escrever requisitos verificáveis.
- Utilizar cenários para comportamentos relevantes.

tasks:

- Dividir a implementação em tarefas pequenas e verificáveis.

operations:

apply:

guidance:

- Executar verificações relevantes antes de concluir.

archive:

guidance:

- Confirmar que a implementação corresponde à especificação.

Esse modelo é apenas um ponto de partida.

Não copie esse conteúdo sem adaptá-lo ao projeto.

30. `config.yaml` versus `AGENTS.md`

É importante não confundir os dois.

`AGENTS.md`

É um documento de instruções para agentes que trabalham no projeto.

Pode conter regras gerais de:

- código;
- arquitetura;
- segurança;
- testes;
- Git;
- utilização do OpenSpec.

`config.yaml`

É uma configuração específica dos workflows do OpenSpec.

Seu `context`, `rules` e `operations` são incorporados ao funcionamento desses workflows.

Uma mesma regra pode aparecer em ambos quando houver uma razão clara, mas **não duplique conteúdo indiscriminadamente**.

31. `specs/` não deve ser preenchida manualmente

Depois da inicialização, você poderá encontrar:

`openspec/specs/`

vazio.

Isso é esperado.

Não crie imediatamente:

`openspec/specs/users/`

openspec/specs/products/

openspec/specs/orders/

apenas porque essas funcionalidades fazem parte do projeto.

As specs representam o comportamento que está sendo especificado e implementado através das changes.

32. O que são as Changes?

Uma **change** representa uma alteração concreta no sistema.

Por exemplo:

Adicionar autenticação

ou:

Permitir exportação de relatórios em PDF

ou:

Implementar recuperação de senha

Uma change não precisa representar necessariamente uma única tabela ou uma única tela.

Ela deve representar uma **mudança funcional ou técnica suficientemente delimitada para ser planejada e verificada**.

33. Primeiro Explore

Antes de criar a primeira change, utilize o workflow:

Explore

O Explore é uma etapa de investigação.

A documentação oficial descreve o Explore como um modo de pensar e investigar uma ideia antes de transformá-la em uma change. Ele pode analisar o código, fazer perguntas, explorar alternativas e desafiar premissas; por padrão, não cria código nem arquivos.

34. O que pedir no primeiro Explore?

Depois de preparar a documentação, o grupo pode solicitar ao agente:

Explore o projeto como um todo.

Leia:

- AGENTS.md
- docs/project-overview.md
- docs/domain-model.md
- openspec/config.yaml

Analise o domínio, a arquitetura, as principais funcionalidades, dependências entre funcionalidades e possíveis riscos.

Identifique uma possível decomposição do projeto em mudanças incrementais.

Não implemente código e não crie uma change ainda.

O resultado deverá ajudar o grupo a decidir:

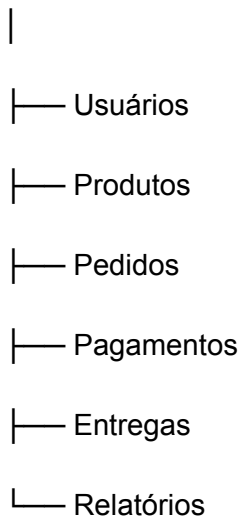
Qual deve ser a primeira mudança a ser implementada?

35. Por que não começar diretamente pelo banco?

O fato de o projeto possuir um modelo de domínio global **não significa que todo o banco deva ser criado imediatamente.**

Imagine:

Modelo conceitual



O agente pode descobrir durante o Explore que uma primeira fatia funcional adequada é:

Usuário



Autenticação



Perfil

e somente depois:

Produto



Pedido

A visão global continua existindo.

O que muda é apenas a ordem da implementação.

36. O que acontece depois do Explore?

Quando o grupo estiver satisfeito com o entendimento do problema, pode solicitar:

Propose

O Propose transforma a ideia em uma change revisável.

No schema **spec-driven**, a change será estruturada em:

proposal.md

specs/

design.md

tasks.md

conforme as dependências definidas pelo schema.

37. Revisar antes de implementar

Nunca trate a saída do agente como automaticamente correta.

O grupo deve revisar:

proposal.md

- O problema está correto?
- O objetivo está claro?
- O escopo está adequado?

spec.md

- O comportamento está correto?
- Os requisitos são verificáveis?
- Há casos de erro importantes?

design.md

- A solução técnica faz sentido?
- Está de acordo com a arquitetura?

tasks.md

- As tarefas são pequenas?
- Todas são necessárias?
- É possível verificar quando uma tarefa está concluída?

Somente depois disso deve-se partir para a implementação.

38. Apply

O **Apply** é a etapa de implementação.

O agente trabalha sobre as tarefas de:

tasks.md

O schema **spec-driven** utiliza essas tarefas como checklist de implementação.

O grupo deve acompanhar a implementação e interromper o agente quando surgir uma decisão que:

- contradiga o domínio;
- altere significativamente a arquitetura;
- exija uma decisão não prevista;
- aumente muito o escopo.

Nessas situações, é preferível **revisar a change** do que simplesmente deixar o agente improvisar.

39. Archive

Quando a change estiver concluída, ela pode ser arquivada.

O OpenSpec utiliza o archive para mover a change concluída para o histórico e atualizar as specs principais com o que foi implementado.

A partir daí:

`openspec/specs/`

passa a refletir o comportamento consolidado do sistema.

E:

`openspec/changes/archive/`

preserva o histórico da mudança.

40. Ciclo completo do desenvolvimento

O processo adotado pelos grupos será:

CONTEXTO

|



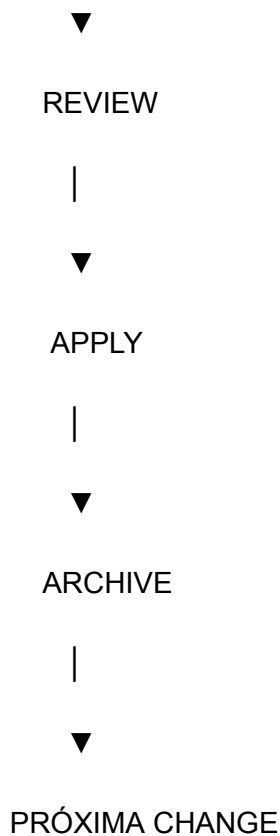
EXPLORE

|



PROPOSE

|



A documentação oficial apresenta exatamente esse ciclo de cinco etapas como o loop principal do OpenSpec.

O **verify** existe como workflow opcional, dependendo do perfil/configuração instalado, e pode ser utilizado quando disponível para verificar a implementação contra o plano.

41. Checklist para o grupo

Antes de iniciar a primeira change, confirme:

Documentação

- ☐ **AGENTS.md** criado.
- ☐ **docs/project-overview.md** criado.
- ☐ **docs/domain-model.md** criado.
- ☐ Os documentos foram revisados pelo grupo.
- ☐ O conteúdo é específico para o projeto do grupo.

Contexto

- ☐ Objetivo do projeto definido.
- ☐ Domínio explicado.
- ☐ Usuários identificados.
- ☐ Principais funcionalidades identificadas.
- ☐ Restrições conhecidas registradas.
- ☐ Arquitetura definida quando já conhecida.
- ☐ Tecnologias obrigatórias registradas.
- ☐ Regras críticas identificadas.

OpenSpec

- ☐ `openspec init` executado.
- ☐ Ferramenta de IA selecionada.
- ☐ `openspec/config.yaml` criado.
- ☐ `schema: spec-driven` configurado.
- ☐ `context` preenchido.
- ☐ `rules` adicionadas somente quando necessárias.
- ☐ `operations` adicionadas somente quando necessárias.
- ☐ `openspec/specs/` criado.
- ☐ `openspec/changes/` criado.

Planejamento

- ☐ Primeiro Explore realizado.
 - ☐ Dependências entre funcionalidades analisadas.
 - ☐ Primeira change identificada.
 - ☐ Primeira change proposta.
 - ☐ Proposal revisado.
 - ☐ Specs revisadas.
 - ☐ Design revisado quando existente.
 - ☐ Tasks revisadas.
-

42. Erros que devem ser evitados

Erro 1 — Pedir para a IA construir tudo

Evite:

"Construa meu sistema completo."

Prefira:

"Explore o projeto e proponha uma decomposição
em mudanças incrementais."

Erro 2 — Criar todo o banco antes de implementar

O modelo conceitual deve ser global.

A implementação física pode ser incremental.

Erro 3 — Colocar tudo no `config.yaml`

O `context` deve conter somente informações relevantes para praticamente todos os workflows.

A própria documentação recomenda manter as regras curtas.

Erro 4 — Fazer um Project Overview superficial

Evite:

"Este sistema será uma aplicação web para resolver X."

O agente precisa de informações suficientes para compreender:

- problema;
 - usuários;
 - objetivos;
 - escopo;
 - funcionalidades;
 - restrições;
 - arquitetura;
 - princípios importantes.
-

Erro 5 — Fazer um Project Overview gigantesco

O documento também não deve conter toda a especificação.

Se uma regra pertence somente a uma funcionalidade, provavelmente ela deverá aparecer na respectiva *spec*.

Erro 6 — Confundir Domain Model com banco

O *domain-model.md* descreve o domínio.

Ele não precisa determinar antecipadamente cada detalhe da implementação física.

Erro 7 — Confiar cegamente no agente

O agente é responsável por auxiliar o desenvolvimento.

O grupo continua responsável pelas decisões do projeto.

O princípio deve ser:

IA propõe



Aluno analisa

↓

Aluno aprova/corrige

↓

IA implementa

43. Estrutura final esperada

Ao terminar a preparação, cada grupo deverá possuir aproximadamente:

meu-projeto/

|

|— AGENTS.md

|

|— docs/

|

| |— project-overview.md

|

| |— domain-model.md

|

|— openspec/

|

| |— config.yaml

|

| |

| |— specs/

|

| |

| |— changes/

| └─ archive/

|

└─ [arquivos da ferramenta de IA]

Os arquivos adicionais dependem da ferramenta de IA escolhida pelo grupo e são instalados pelo próprio `openspec` `init`.

44. O que deve existir dentro de cada documento?

Como resumo:

AGENTS.md

|

└─ Como a IA deve trabalhar

docs/project-overview.md

|

└─ O que é o projeto

docs/domain-model.md

|

└─ Quais são os conceitos e relacionamentos do domínio

openspec/config.yaml

|

└─ schema

|— context

|— rules

|— operations

openspec/specs/

|

|— O comportamento consolidado do sistema

openspec/changes/

|

|— Mudanças atualmente sendo planejadas/implementadas

openspec/changes/archive/

|

|— Histórico das mudanças concluídas

45. Regra de ouro

Ao preparar o projeto para o agente, não tente responder antecipadamente a todas as perguntas de implementação.

Procure responder primeiro:

O que estamos construindo?

Qual problema estamos resolvendo?

Quem utiliza o sistema?

Quais são os conceitos fundamentais?

Quais restrições precisam ser respeitadas?

Quais decisões arquiteturais já estão definidas?

Como queremos que o agente trabalhe?

Depois, utilize o OpenSpec para responder, change por change:

O que vamos construir agora?

Como deve funcionar?

Como será implementado?

Como saberemos que está correto?

Essa separação entre **contexto global** e **mudanças incrementais** é a base da metodologia adotada para o projeto.